

Retrieval Systems: A Short Primer

A sample document bundled with doc-agent, written for the demo so that no third-party licence applies to it.

1. Why retrieval exists

A language model answers from weights fixed at training time. That creates four practical gaps. Its knowledge is frozen at a cutoff date. It has never seen private material such as internal wikis, contracts, or ticket histories. When it does not know something it tends to produce a fluent guess rather than an admission. And its context window puts a hard ceiling on how much text can be supplied per request, so pasting an entire manual into every prompt is not an option.

Retrieval-augmented generation closes all four at once. Rather than asking the model to recall a fact, the system searches a corpus at question time and supplies the passages it found as part of the prompt. The model's job narrows from remembering to reading. Facts change by reindexing the corpus, not by retraining.

The trade is that answer quality now depends on retrieval quality. A model given the wrong three paragraphs will answer confidently from them. Most production failures in these systems are retrieval failures wearing a generation costume.

2. Tokenization and why chunk size is measured in tokens

Models do not read characters or words. Text is split into tokens by a learned vocabulary, and both pricing and context limits are expressed in those units. English prose averages roughly four characters per token, but the variance is large: dense technical vocabulary, code, identifiers, and non-Latin scripts all tokenize far less efficiently than plain narrative text.

This is why chunk sizes given in characters are unreliable. A thousand characters of casual prose might be 200 tokens, while a thousand characters of code or chemical nomenclature might be 400. A pipeline that budgets in characters therefore either wastes context or overflows it, and which one happens depends on the document.

Counting tokens directly removes the guesswork. A tokenizer is cheap to run, so measure the real number at ingestion and again when assembling a prompt.

3. Chunking strategy

A document cannot be embedded as one vector and searched usefully. A single vector for fifty pages describes everything and matches nothing in particular. Documents are therefore split into passages, each embedded separately, so a search can return the specific paragraph that answers a question.

Chunk size trades precision against completeness. Small chunks match narrowly but may omit the sentence that gives the answer its meaning. Large chunks carry their context but dilute the embedding, so the match becomes vague. A few hundred tokens per chunk is a reasonable starting point for prose.

Overlap exists to protect answers that straddle a boundary. Without it, a definition split across two chunks may be retrievable from neither. Ten to twenty percent overlap is usually enough; forty percent mostly buys redundant neighbours and a larger index.

Split on structure where the document offers it. Paragraph and section boundaries are meaningful, and respecting them produces chunks that read as coherent passages rather than fragments ending mid-sentence.

4. Embeddings

An embedding model maps text to a fixed-length vector positioned so that passages with similar meaning sit near one another. Similarity is then a geometric question, most often cosine distance, which compares direction and ignores magnitude.

Because meaning is compared rather than wording, a question phrased differently from the document can still retrieve the right passage. This is the property that keyword search lacks and the reason embeddings are worth the extra machinery.

One rule admits no exceptions: the index and the query must use the same embedding model. Vectors from different models occupy unrelated spaces, and mixing them does not raise an error. It silently returns confident nonsense, which is considerably worse than a crash.

5. Vector databases

A vector database answers one question quickly: given this vector, which stored vectors are closest? Comparing against every stored vector is exact but scales linearly, which stops being viable well before a million passages.

Approximate nearest neighbour indexes solve this. HNSW, the most common, builds a navigable graph in which search hops toward closer neighbours rather than scanning. Two parameters dominate its behaviour: the graph's connectivity, and how much effort is spent during construction. Both trade index build time and memory against recall.

Approximate means what it says. These indexes occasionally miss a true nearest neighbour, and in exchange they are orders of magnitude faster. For retrieval feeding a language model that trade is almost always correct, since the model receives several passages and a single missed candidate rarely changes the answer.

Store the original text alongside each vector. A vector alone cannot be shown to a user or placed in a prompt, and provenance such as page numbers is what makes an answer checkable.

6. Agents over single-pass retrieval

The simplest pipeline embeds the question, retrieves the closest passages, and generates an answer. It works when the question's wording resembles the document's and when one lookup suffices.

It fails in two common cases. A question with several parts needs several searches. And a question phrased in the user's vocabulary rather than the document's may retrieve nothing relevant on the first attempt.

An agent addresses both by keeping the model in the loop. It chooses a search query, reads what came back, judges whether the material answers the question, and searches again with different terms if not. The cost is more model calls per question and a need for limits: without a step ceiling, a model that never converges loops until something stops it.

7. Operational concerns

Ingestion is slow relative to a web request. Reading a large PDF, splitting it, and embedding every chunk takes seconds to minutes, which is too long to hold a connection open. Production systems push that work onto a queue and let a worker process consume it, so the request returns immediately with a job identifier.

Queues change the deployment shape. Once a worker runs in its own process, it can no longer reach an index held in the web process's memory, so the vector store must become a service both can address. This is a common progression: the in-process store that made development simple is the first thing to go when work moves to a background worker.

Rate limits and provider outages are ordinary conditions, not exceptions. Every hosted model API returns 429 when a quota is spent and 5xx when it is overloaded. Distinguish the limits that clear on their own from those that do not: retrying a per-minute limit is correct, and retrying a daily quota only wastes the time before failing anyway.

Finally, measure memory. Embedding runtimes size their allocation arenas to the batches they are given, so batch size is often the largest single influence on peak usage, and peak usage is what decides whether a container survives its memory limit.